

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Best Bike Paths

Implementation and Testing Document

Lorenzo Fiamingo
Alexander Gabriel Fox
Foad Zahedi Labaf

Academic year 2025/2026

Contents

Contents **iii**

List of Tables **vii**

1 Introduction **1**

1.1 Purpose 1

1.2 Scope 1

1.3 Definitions, Acronyms, Abbreviations 2

1.3.1 Definitions 2

1.3.2 Acronyms 3

1.3.3 Abbreviations 3

1.4 Version History 3

1.5 Reference Documents 3

1.6 Document Structure 3

2 Requirement and Functions **5**

2.1 Overview of Implemented Requirements 5

2.1.1 **User Management Functionalities** 5

2.1.2 **Path Recording and Storage Functionalities** 5

2.1.3 **Path Visualization and Management** 6

2.1.4 **Community Path Browsing and Detail Functionalities** 6

2.1.5 **Permission Management** 7

2.2 Implemented Requirements 7

3 Development Frameworks **11**

3.1 Frameworks 11

3.1.1 Frontend 11

3.1.2 Backend 12

3.2 Programming Languages 13

3.2.1 Frontend 13

3.2.2 Backend 13

4 Source Code Structure **15**

4.1 iOS Client Structure 15

4.2 Firebase Backend (Functions Environment) 16

4.3 Dependency Management 16

5 Testing **17**

5.1 Unit Testing 18

5.1.1 Backend Business Logic 18

5.1.2 Mocking Strategy and Isolation 18

5.1.3 Unit Test Cases 18

5.1.4 **Outcome Considerations** 26

5.2 Integration Testing 27

5.2.1 Backend Integration Logic 27

5.2.2 Mocking Strategy and Isolation 27

5.2.3 Integration Test Cases 27

5.3 System Testing 29

5.3.1 System Test Cases 29

5.3.2 **Outcome Considerations** 32

6 Installation Instructions **33**

6.1 Option 1: Public Link 33

6.2 Option 2: Internal Testing (Manual Addition) 33

6.3 Prerequisites 33

7 Available acceptance testing infrastructure **35**

7.1 Physical iOS Devices 35

7.2 iOS Simulator (macOS) 35

7.2.1 Comparison Summary 35

8 Effort Spent **37**

9 References **39**

List of Tables

7.1 Comparison between Physical Device and macOS Simulator. 35
8.1 The number of hours each group member has worked for this document. . . 37

1 Introduction

1.1 Purpose

This document describes the implementation and integration testing of the BestBikePaths system.
The system has been developed in accordance with the requirements defined in the Requirement Analysis and Specification Document (RASD) and the architectural choices presented in the Design Document (DD).
The purpose of this document is to explain how those requirements and design decisions are reflected in the actual software implementation and how the different system components are integrated and verified. In particular, the document focuses on the interaction between the mobile application, the backend services, and the external components involved in the system execution.
The document presents the main implemented functionalities, the technologies adopted during development, and the integration testing activities carried out to validate component interactions. These tests aim to ensure that the system behaves consistently with the expected scenarios described in the RASD and that the architectural solutions defined in the DD are correctly realized.
No new requirements or design decisions are introduced in this document.

1.2 Scope

This document focuses exclusively on the implementation and integration testing activities carried out for the BestBikePaths system.
The attention is mainly on the backend part of the application and on how it interacts with the mobile client during normal system execution.

Only the parts of the system that have been effectively developed are considered. For this reason, the document focuses on the functionalities that are currently available and on the technologies that have been used to implement them. The interaction between the mobile application and the backend services is a central aspect of our analysis, since several system functionalities depend on this communication.

The testing activities described here concern the integration of different components, their interaction, and their internal implementation.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

User: Term used to indicate both authenticated and unauthenticated individuals who interact with the application.

Authenticated User: User who is authenticated in their BBP account. An authenticated user has full access to the functionalities offered by the application.

Unauthenticated User: Users who do not have a BBP account, or who have an account but aren't currently authenticated within the system. These users have limited access to the application's functionalities.

Bike Path: A bike path is a collection of information associated with a specific route. This information may refer to a single user trip and, in that case, includes performance-related data describing the user's ride, as well as qualitative information about the route itself. The term bike path is also used to indicate the aggregated information derived from multiple user-specific bike paths that follow the same route. This information will therefore exclude individual performance data and primarily consists of aggregated data describing the overall quality of the route.

Bike Trip: A bike trip indicates a cycling session that may optionally be recorded by the application at the cyclist's discretion. During the trip, data is collected from the device's GPS and motion sensors and may be enriched with meteorological information.

Motion Sensors: This term refers to the user's device's hardware components that allow for the collection of performance-related and qualitative data about the bike trip. Specifically, the gyroscope of the device is responsible for measuring angular velocity and is used to identify obstacles along the path. The accelerometer is responsible for measuring the acceleration of the user during the trip and contributes both to obstacle detection and to the computation of the user's velocity.

SMTP Server: An SMTP server, or email service provider, is an external system responsible for sending transactional emails on behalf of the application.

Status: Term used to indicate the overall assessment of a path's quality, which can be either manually specified by the user or computed by the system based on the frequency of obstacles along the path.

1.3.2 Acronyms

API: Application Programming Interface
BBP: Best Bike Paths
GPS: Global Positioning System
HTTPS: HyperText Transfer Protocol Secure
JSON: JavaScript Object Notation
MVVM: Model-View-ViewModel
NPM: Node Package Manager
REST: Representational State Transfer
SPM: Swift Package Manager
SMTP: Simple Mail Transfer Protocol
Wi-Fi: Wireless Fidelity

1.3.3 Abbreviations

R: Requirement
UT: Unit Test
IT: Integration Test
ST: System Test

1.4 Version History

- Version 1: 01/02/2026

1.5 Reference Documents

The following reference materials were used for the preparation of this document:

- The Requirement Analysis and Specification Document.
- The Design Document
- The specification for the 2025/26 Implementation & Testing for the Software Engineering II course.
- The WeBeep slides of the Software Engineering II course.
- Apple Inc., **Apple Human Interface Guidelines**, online documentation.
- Google LLC, **Material Design Guidelines**, online documentation.
- Apple Inc., **Core Motion Framework Documentation**, online documentation.
- Apple Inc., **Core Location Framework Documentation**, online documentation.
- Apple Inc., **WeatherKit Framework Documentation**, online documentation.

1.6 Document Structure

1. **Introduction**: A brief overview of the contents of the document and its purpose.
2. **Requirement and Functions**: Illustrates the implemented functions and the requirements that they fulfill.

3. **Development Frameworks**: Describes the development frameworks and programming languages adopted for the project, along with a discussion of the advantages and disadvantages associated with these choices.
4. **Source Code**: Explains the structure of the backend and frontend source code.
5. **Testing**: Describes the test cases that were written for testing the application's functionalities.
6. **Installation Instructions**: Instructions for installation and deployment of the application.
7. **Effort spent**: Includes the amount of time dedicated by each member for the realization of each section of the document.
8. **References**: Reference to documents or tools used for writing this document.

2 Requirement and Functions

2.1 Overview of Implemented Requirements

The current version of the BestBikePaths system implements most of the requirements defined in the Requirements Analysis and Specification Document.
The implemented functionalities cover the core objectives of the application and allow users to interact with the system through the mobile client and the backend services.

This section describes the functionalities that are available in the system at the time of writing, while the requirements are discussed separately

2.1.1 User Management Functionalities

The BestBikePaths system includes user management functionalities that allow users to access the application and interact with the available services. Users can create an account, authenticate themselves, and remove their account through the mobile application.

User-related operations are handled by the backend services and are exposed to the mobile client through dedicated interfaces.
This design choice allows the system to associate stored data, such as bike paths, with the correct user and to enforce basic access control rules.

User management functionalities have been implemented because they are required to support most of the use cases defined in the RASD, including personal path management and access to community-related features.

2.1.2 Path Recording and Storage Functionalities

The system allows users to create and store bike paths using the mobile application.
In the current implementation, path creation includes manual path insertion, which enables users to explicitly define a route and save it in the system, and automatic recording, which allows users to start recording sessions for the automatic collection of data related to the followed path.

Once a path is created by the user or the recording session ends, the corresponding data is sent to the backend services, where it is stored and associated with the user's account.
The backend is responsible for managing the persistence of path data and for maintaining the link between paths and users.

This functionality has been implemented because it represents a fundamental requirement of the BestBikePaths system.

2.1.3 Path Visualization and Management

The BestBikePaths system allows users to visualize and manage the bike paths stored in their personal collection.
Through the mobile application, users can retrieve previously saved paths and display them on the map.

In addition to visualization, users can manage their stored paths by modifying their associated privacy level.
These operations are supported by the backend services and ensure that each user can only access and modify paths that belong to their account.

Path visualization and management functionalities have been implemented to allow users to review and organize their personal paths, supporting one of the main use cases of the application.

2.1.4 Community Path Browsing and Detail Functionalities

The system also supports functionalities related to the exploration of bike paths shared by other users.
Community path browsing allows users to view paths that have been made available to the community and to optionally select one to follow during an automatic recording session.

Through the mobile application, users can browse the available community paths and select a specific path in order to access its detailed information.
The path detail view provides additional data about the selected path and allows users to better understand the characteristics of the route.

These functionalities have been implemented to support the social aspect of the BestBikePaths system, as described in the requirements analysis.
They allow users to benefit from paths created by other members of the community and represent an important extension of the personal path management features.

2.1.5 Permission Management

When the system needs a capability permission for a feature, such as the user location to record a bike path, a sheet modal is shown. If the capability is available and the user never denied the authorization, and decides to give their permission, the system dialog to give permission is shown, and the user can allow the app to use the capability. If the capability is available and the user denied the authorization, but then decides to give their permission, they must navigate to the settings and change the access authorization status manually. If the capability is unavailable, the user can just close the modal and not use the app feature.

2.2 Implemented Requirements

Account Management

R1: The system shall allow an unauthenticated user to create an account.
Implemented

R2: The system shall allow an unauthenticated user to sign into the app.
Implemented

R3: The system shall allow an authenticated user to sign out from the app.
Implemented

R4: The system shall allow an authenticated user to delete their account.
Implemented

Permission Management

R5: The system shall request the user's permission to use a device capability.
Implemented

R6: The system shall properly handle user permission denial.
Implemented

Automatic Bike Path Recording

Start Recording
R7: The system shall allow the authenticated user to start the automatic recording mode of a bike path.
Implemented

R8: The system shall use the GPS to track the user's real-time position during the trip.
Implemented

R9: The system shall use the device's sensors to collect information about obstacles along the path and the speed of the user.
Implemented

R10: The system shall display the current trip data (current duration time, map of the path, average speed, top speed, weather conditions, distance covered) while the user is biking.
Implemented

R11: The system shall prompt the user for confirmation of a detected obstacle while biking.
Implemented In App (System Notifications Not Implemented)

R12: The system shall allow the authenticated user to insert an obstacle manually.
Implemented

Stop Recording
R13: The system shall allow the authenticated user to manually stop the automatic recording of a bike path.
Implemented

R14: The system shall stop collecting GPS data once the authenticated user selects "Stop Recording".
Implemented

R15: The system shall stop collecting sensor data once the authenticated user selects "Stop Recording".
Implemented

R16: The system shall compute the status for the recorded path based on the frequency of obstacles.
Implemented

R17: The system shall store the path and its associated status in the authenticated user's personal history.
Implemented

R18: The system shall cancel the recording if the accuracy of the GPS data drops under a certain threshold (over 10 meters for more than 15 seconds).
Not implemented

Pause and Resume
R19: The system shall automatically pause recording when the authenticated user stops biking.
Implemented

R20: The system shall automatically resume recording when the authenticated user starts biking.
Implemented

R21: The system shall allow the user to manually pause the recording.
Implemented

R22: The system shall allow the user to manually resume the recording.
Implemented

Manual Bike Path Insertion

R23: The system shall allow the authenticated user to manually create a path by inserting a starting point, destination, intermediary stops and defining privacy and a quality status.
Implemented

Personal History Management

User Bike Path List
R24: The system shall display a list of the user's previously recorded paths in chronological order when accessing their history page. *Implemented*

User Bike Path Detail
R25: The system shall display all the collected data of a personal path when the user visits its dedicated page.
Implemented

R26: The system shall allow the user to set the privacy status of the path.
Implemented

R27: The system shall allow the user to delete their personal bike paths.
Implemented

Community Bike Path Browsing

R28: The system shall allow the user to browse publicly available bike paths by specifying a starting point and a destination.
Implemented

R29: The system shall show the user the matching paths, sorting them by their system-computed average quality score (calculated based on the public user path instances).
Implemented

R30: The system shall display for each path the route map along with the system-computed quality score.
Implemented

R31: The system should display an alert badge if a path contains incompatible roads for biking.
Implemented

R32: The system shall allow the user to select a path as the suggested route for a new automatic recording session.
Implemented

3 Development Frameworks

The development of the BestBikePaths system required the use of different technologies for the mobile client and for the Backend services.
The adopted frameworks and platforms follow the design decisions described in the Design Document and were selected mainly to simplify development and integration activities..

During the implementation phase, additional frameworks (not initially outlined in the previous documents) were integrated to enhance the robustness of the core logic and ensure overall code stability.
The choices described in this section reflect what was effectively used while developing and integrating the system components.

3.1 Frameworks

3.1.1 Frontend

Swift UI
The user interface was built using SwiftUI, Apple's declarative framework to build user interfaces across all Apple platforms. By defining the UI using it's MVVM model, the app maintains a reactive flow where the view updates automatically in response to data changes, significantly reducing the glue code typically required in UIKit.

SwiftUI-AsyncButton
A specialized Swift Package (installed via SPM) that extends standard button functionality to natively support asynchronous operations. It simplifies the UI lifecycle by automatically managing loading states and preventing redundant triggers during concurrent executions.

SwiftUI-Introspect
A Swift Package used to access the underlying UIKit components of SwiftUI views. Specifically, it's here used to intercept the *UISheetPresentationController* to implement

advanced customizations for modal sheets such as precise detent control and gestures that are central to the BBP app's user experience.

Firebase iOS SDK
The core suite used to bridge the mobile client with backend services. It facilitates secure user authentication via Firebase Auth and provides the communication layer for invoking Cloud Functions, ensuring seamless data flow between the iOS app and the serverless environment.

MapKit
MapKit was integrated to handle all geo-spatial requirements. It provides the core mapping interface, handles user location tracking, and renders the polyline paths that represent bike paths. It also provide the Directions API, that can be used to retrieve detailed step-by-step navigation and route data based on start and end points.

WeatherKit
WeatherKit was integrated to retrieve current weather data during user path recording.

CoreMotion
Used to obtain device motion data (accelerometers and gyros) and user activity detection.

CoreLocation
Used to obtain user location and speed data.

3.1.2 Backend

Firebase
The application's backend was implemented using Firebase Cloud Functions, deployed on Firebase's serverless infrastructure. The cloud functions interact with the Cloud Firestore database via the Firebase Admin SDK, which ensures secure and privileged access to the project's data without exposing credentials to the client. User Authentication is handled by Firebase Auth that enables secure access control and identity verification, ensuring that each user can only interact with their own data or authorized community resources through unique, cryptographically signed tokens.

Geofire-Common
A specialized framework within the Firebase ecosystem, integrated to perform proximity searches in Firestore by leveraging geohashing for efficient spatial querying.

Turf.js
A GeoJSON handling framework we adopted to enhance the robustness of path similarity algorithms

Nodemailer
An email sending library for Node.js used for connecting to the app SMTP provider, SendGrid.

3.2 Programming Languages

3.2.1 Frontend

Swift
Swift is a strongly typed language mostly used for software development on Apple platforms. It's known for its performance, safety, and modern syntax.
Working with Swift made it possible to directly use iOS frameworks and tools, simplifying the development of the interface.
However, this native approach initially tied the application strictly to the Apple ecosystem. While the UI must still be developed in Kotlin to meet Android's specific standards, the fact that Swift is now a multi-platform language changes the outlook for the future.
This evolution will make it significantly easier to extend support to Android by sharing core logic, allowing us to scale the project more efficiently while still retaining all the performance and safety advantages of a truly native application.

3.2.2 Backend

TypeScript
The backend is implemented in TypeScript, which is used to write the functions that communicate with the mobile application.
These functions manage user and bike path data and handle requests coming from the client.
Using TypeScript helped keep the Backend code clean and organized during development, and the structure of the exchanged data was easier to control compared to JavaScript.
Furthermore, it allowed us to catch several errors at compile time.
An additional build step was required compared to plain JavaScript, but this did not affect the overall development flow.

4 | Source Code Structure

The project is organized into two primary directories, which are structured to function as two separate Git repositories in a production environment: the **iOS Client** (`Implementation/swift-bbp-client`) and the **Firebase Backend** (`Implementation/firebase-bbp-backend`). This separation ensures a clean decoupling of concerns, where the mobile app handles the user interface and sensor data, while the cloud infrastructure manages data persistence and heavy geo-spatial calculations.

4.1 iOS Client Structure

The Xcode project is organized into logical modules (containing data models, data managers, data sources, UI components) that are separated by feature. This structure facilitates modular development and clear separation of concerns. The main ones are:

- **User**: Manages the user life cycle, from identity models (`User`) to authentication and account management interfaces.
- **UserPath**: Dedicated modules for path data structures and user-specific route history, ensuring that the geographic logic is decoupled from the UI.
- **CommunityPath**: Handles all features related to shared routes. It includes the data sourcing logic (`CommunityPathSource`) and the UI components for searching and displaying community-contributed paths.
- **CapabilityAccess**: Manages the integration with hardware and system services. It includes `LocationManager`, `MotionManager`, acting as a bridge between Apple's native frameworks and the app's business logic.
- **Content**: The core UI container of the application. It hosts the `ContentMap`, the main `ContentView`, and the `Coordinator` pattern used to manage complex interactions between the map and the interface.
- **Extension**: Contains Swift extensions used to augment existing types. Notable examples include custom error handling (`AnyLocalizedError`) and UI state extensions for asynchronous operations.
- **Modifiers, LocalSearch**: Specialized folders for custom SwiftUI view modifiers (like `SheetInsetting`) and location search auto-completion features.

We defined for each feature specific protocols (such as UserSource, UserPathSource, and CommunityPathSource). This architectural choice allows for multiple implementations, enabling us to seamlessly switch between TestUserSource for testing and FirebaseUserSource for production through dependency injection.

4.2 Firebase Backend (Functions Environment)

The backend logic is centralized within the `functions/` directory, structured to facilitate development, local emulation, and automated testing:

- **src**: The TypeScript source code.
  - **index.ts**: The entry point where Cloud Functions (HTTPS and Triggers) are exported.
  - **managers**: Module-specific logic (e.g., `pathManager.ts`, `userManager.ts`) that isolates business logic from the Firebase transport layer in `index.ts`.
  - **constants**: Shared constant values providing a single source-of-truth.
- **___tests___**: A comprehensive test suite using Jest. It contains unit tests for the manager modules, using mocked environments for Firestore and Turf.js to ensure logic reliability.
- **package.json**: Manages dependencies and defines scripts for linting, compiling, testing and deploying the functions.

4.3 Dependency Management

The project relies on two main package managers:

- **SPM (Swift Package Manager)**: Used for all iOS third-party libraries, integrated directly into Xcode.
- **NPM (Node Package Manager)**: Used for the backend to manage TypeScript, Firebase Admin SDK, and geo-spatial libraries.

5 | Testing

This section outlines the verification strategies employed to ensure the application's reliability, from individual logic components to the complete end-to-end user flow. Testing activities were performed to verify the correct behavior of the BestBikePaths system with respect to the implemented requirements.

All tests described in this section were executed on the implemented version of the system.

5.1 Unit Testing

5.1.1 Backend Business Logic

The backend business logic, implemented via Firebase Cloud Functions, was validated using the Jest framework. The primary objective was to verify the "Managers" in isolation, ensuring that geo-spatial algorithms and data processing rules remain accurate regardless of external infrastructure status.

5.1.2 Mocking Strategy and Isolation

Due to the serverless nature of the backend, unit tests do not interact with live Firestore or Firebase Auth instances. Instead, the following mocking strategy was adopted to simulate the environment:

Dependency Mocking
By utilizing `jest.mock()`, complex external modules such as `firebase-admin` and `@turf/turf` were replaced with stub implementations that return predictable data.

Database Simulation
The hierarchical nature of Firestore queries (e.g., `collection().where().get()`) was reconstructed using nested Jest mocks. This allows the suite to simulate various database states easily (such as empty results, multiple document returns, or execution errors).

Temporal Control
To validate time-sensitive logic (e.g., filtering out community reports older than a year), the Timestamp object was mocked to simulate specific points in time, ensuring the filtering logic behaves as expected.

Firestore Access
To test Firestore Security Rules, we used `@firebase/rules-unit-testing` in order to to verify that our access control logic correctly allows or denies requests based on a user's authentication state. The framework permits the creation a test environment that makes it possible to call Firestore locally.

5.1.3 Unit Test Cases

Community Path Manager

getCommunityPaths
UT1: Should return community paths with calculated status

- **Precondition**: User paths exist with validity votes and associated community paths.
- **Action**: Request community paths.

- **Expected result**: Returns a list of paths with the correct aggregated status score.
- **Outcome**: *Passed*

UT2: Should sort paths by status descending
- **Precondition**: Multiple community paths exist with different calculated status scores (e.g., 5 and 3).
- **Action**: Request community paths.
- **Expected result**: The returned list is ordered with the highest status paths first.
- **Outcome**: *Passed*

UT3: Should exclude paths with no public reports
- **Precondition**: A path exists but has no associated public user reports.
- **Action**: Request community paths.
- **Expected result**: The path without public reports is excluded from the result.
- **Outcome**: *Passed*

calculateCommunityPathStatus
UT4: Should return 0 if no valid reports
- **Precondition**: An empty list of reports is provided.
- **Action**: Calculate status.
- **Expected result**: Returns a status of 0.
- **Outcome**: *Passed*

UT5: Should aggregate votes correctly
- **Precondition**: A mix of positive and negative votes exists for a path.
- **Action**: Calculate status.
- **Expected result**: Returns the correct aggregated score (e.g., average of majority).
- **Outcome**: *Passed*

UT6: Should ignore old reports
- **Precondition**: Reports list contains both recent reports and reports older than the validity threshold.
- **Action**: Calculate status.
- **Expected result**: Old reports are ignored in the calculation.
- **Outcome**: *Passed*

UT7: Should return average winners in case of a tie
- **Precondition**: Reports result in a tie for the most frequent status.
- **Action**: Calculate status.
- **Expected result**: Returns the average of the tied winning statuses.
- **Outcome**: *Passed*

Path Manager

distance
UT8: Should calculate distance correctly

- **Precondition**: Two geographic coordinates are provided.
- **Action**: Calculate the distance between them.
- **Expected result**: Returns the correct distance in meters.
- **Outcome**: *Passed*

arePathsSimilar
UT9: Should return true for identical paths

- **Precondition**: Two identical paths (same coordinates) are provided.
- **Action**: Check for similarity.
- **Expected result**: Returns true.
- **Outcome**: *Passed*

UT10: Should return false for very different paths

- **Precondition**: Two spatially distinct paths are provided.
- **Action**: Check for similarity.
- **Expected result**: Returns false.
- **Outcome**: *Passed*

savePath
UT11: Should save path to firestore

- **Precondition**: A valid path object with sufficient points is provided.
- **Action**: Save the path.
- **Expected result**: The path is successfully written to the 'paths' Firestore collection.
- **Outcome**: *Passed*

UT12: Should throw error for path with < 2 points

- **Precondition**: A path with only one point is provided.
- **Action**: Attempt to save the path.
- **Expected result**: An error is thrown indicating insufficient points.
- **Outcome**: *Passed*

getSimilarPath
UT13: Should return null if no similar path found

- **Precondition**: No existing path in the database matches the input path's geometry.
- **Action**: Search for a similar path.
- **Expected result**: Returns null.
- **Outcome**: *Passed*

UT14: Should return path if similar path found

- **Precondition**: A path exists in the database that is geometrically similar to the input.
- **Action**: Search for a similar path.
- **Expected result**: Returns the matching existing path object.
- **Outcome**: *Passed*

User Path Manager

saveUserPath
UT15: Should save user path and link to existing path if found

- **Precondition**: A similar community path already exists for the user's path.
- **Action**: Save the user path.
- **Expected result**: The user path is saved with a reference (`pathId`) to the existing community path.
- **Outcome**: *Passed*

UT16: Should create new path if no similar path found

- **Precondition**: No similar community path exists.
- **Action**: Save the user path.
- **Expected result**: A new community path is created, and the user path is saved linking to it.
- **Outcome**: *Passed*

getUserPaths
UT17: Should return user paths

- **Precondition**: User has saved paths in the database.
- **Action**: Retrieve paths for the user.
- **Expected result**: Returns a list of the user's paths.
- **Outcome**: *Passed*

updateUserPathStatus
UT18: Should update status if owner

- **Precondition**: The requesting user is the owner of the path.
- **Action**: Update the path status.
- **Expected result**: The status update is successfully applied.
- **Outcome**: *Passed*

UT19: Should throw error if not owner

- **Precondition**: The requesting user is not the owner of the path.
- **Action**: Attempt to update the path status.
- **Expected result**: An error is thrown denying the update.
- **Outcome**: *Passed*

deleteUserPath
UT20: Should delete user path and cleanup orphan path

- **Precondition**: A user path exists and is the only reference to a community path.
- **Action**: Delete the user path.
- **Expected result**: The user path is deleted, and the orphaned community path is also removed.
- **Outcome**: *Passed*

User Manager

createUser
UT21: Should create a user and return a custom token when email is verified

- **Precondition**: A valid email verification code exists for the user.
- **Action**: Create a new user account.
- **Expected result**: User is created in Auth and a custom session token is returned.
- **Outcome**: *Passed*

UT22: Should throw error if email is not verified

- **Precondition**: No valid email verification code exists.
- **Action**: Attempt to create a user account.
- **Expected result**: Creation fails with an error.
- **Outcome**: *Passed*

checkUserExists
UT23: Should return true if user exists

- **Precondition**: A user with the given email exists in Auth.
- **Action**: Check if user exists.
- **Expected result**: Returns true.
- **Outcome**: *Passed*

UT24: Should return false if user does not exist

- **Precondition**: No user with the given email exists.
- **Action**: Check if user exists.
- **Expected result**: Returns false.
- **Outcome**: *Passed*

checkEmailVerificationCode
UT25: Should return false if code is expired

- **Precondition**: The verification code in the database has passed its expiration time.
- **Action**: Validate the code.
- **Expected result**: Returns false.
- **Outcome**: *Passed*

UT26: Should return true if code is valid and not expired

- **Precondition**: The verification code is correct and within the validity period.
- **Action**: Validate the code.

- **Expected result**: Returns true.
- **Outcome**: *Passed*

sendEmailVerificationCode
UT27: Should send an email and save code to firestore
- **Precondition**: A valid email address is provided.
- **Action**: Request verification code.
- **Expected result**: An email is sent via nodemailer and the code is stored in Firestore.
- **Outcome**: *Passed*

signIn
UT28: Should return custom token on successful sign in
- **Precondition**: Valid credentials are provided.
- **Action**: Sign in.
- **Expected result**: Returns a custom authentication token.
- **Outcome**: *Passed*

UT29: Should throw HttpsError on failed sign in
- **Precondition**: Invalid credentials are provided.
- **Action**: Attempt to sign in.
- **Expected result**: Throws an HttpsError.
- **Outcome**: *Passed*

deleteUser
UT30: Should delete user given an email
- **Precondition**: User exists in the system.
- **Action**: Delete the user.
- **Expected result**: The user account is removed from Firebase Auth.
- **Outcome**: *Passed*

Function Triggers

createUser
UT31: Should call userManager.createUser
- **Precondition**: The `createUser` callable function is triggered.
- **Action**: Execute the trigger.
- **Expected result**: Delegates execution to `userManager.createUser`.
- **Outcome**: *Passed*

getCommunityPaths
UT32: Should call communityPathManager.getCommunityPaths
- **Precondition**: The `getCommunityPaths` callable function is triggered.
- **Action**: Execute the trigger.

- **Expected result**: Delegates execution to `communityPathManager.getCommunityPaths`.
- **Outcome**: *Passed*

saveUserPath
UT33: Should call userPathManager.saveUserPath

- **Precondition**: The `saveUserPath` callable function is triggered by an authenticated user.
- **Action**: Execute the trigger.
- **Expected result**: Delegates execution to `userPathManager.saveUserPath`.
- **Outcome**: *Passed*

UT34: Should throw if unauthenticated

- **Precondition**: The `saveUserPath` callable function is triggered without authentication.
- **Action**: Execute the trigger.
- **Expected result**: Throws an unauthenticated error.
- **Outcome**: *Passed*

Firestore Security Rules

Users Collection
UT35: Should allow user to write their own profile

- **Precondition**: Authenticated user writes to their own document.
- **Action**: Write to `users/{userId}`.
- **Expected result**: Write allowed.
- **Outcome**: *Passed*

UT36: Should allow user to read their own profile

- **Precondition**: Authenticated user reads their own document.
- **Action**: Read `users/{userId}`.
- **Expected result**: Read allowed.
- **Outcome**: *Passed*

UT37: Should not allow user to write others' profile

- **Precondition**: Authenticated user writes to another user's document.
- **Action**: Write to `users/{otherId}`.
- **Expected result**: Write denied.
- **Outcome**: *Passed*

UT38: Should not allow user to read others' profile

- **Precondition**: Authenticated user reads another user's document.
- **Action**: Read `users/{otherId}`.
- **Expected result**: Read denied.
- **Outcome**: *Passed*

User Paths Collection

UT39: Should allow owner to create path with matching userId

- **Precondition**: User creates a path where `userId` matches their Auth UID.
- **Action**: Create document in `user_paths`.
- **Expected result**: Create allowed.
- **Outcome**: *Passed*

UT40: Should not allow owner to create path with mismatching userId

- **Precondition**: User creates a path where `userId` does not match their Auth UID.
- **Action**: Create document in `user_paths`.
- **Expected result**: Create denied.
- **Outcome**: *Passed*

UT41: Should allow owner to read their path

- **Precondition**: User reads a path document they own.
- **Action**: Read document from `user_paths`.
- **Expected result**: Read allowed.
- **Outcome**: *Passed*

UT42: Should allow others to read public paths

- **Precondition**: User reads a path marked `isPublic: true`.
- **Action**: Read document from `user_paths`.
- **Expected result**: Read allowed.
- **Outcome**: *Passed*

UT43: Should not allow others to read private paths

- **Precondition**: User reads a path marked `isPublic: false` owned by someone else.
- **Action**: Read document from `user_paths`.
- **Expected result**: Read denied.
- **Outcome**: *Passed*

Paths (Community) Collection

UT44: Should allow read for authenticated users

- **Precondition**: Authenticated user requests a community path.
- **Action**: Read `paths/{pathId}`.
- **Expected result**: Read allowed.
- **Outcome**: *Passed*

UT45: Should deny write for users

- **Precondition**: Authenticated user attempts to modify a community path.
- **Action**: Write to `paths/{pathId}`.
- **Expected result**: Write denied.
- **Outcome**: *Passed*

5.1.4 Outcome Considerations

All unit tests passed successfully, achieving a code coverage of over 90% across core modules ensuring that every non trivial logical branch (if/else) and error handler (catch) has been executed and verified.

5.2 Integration Testing

5.2.1 Backend Integration Logic

The purpose of the following integration tests is to verify that the backend software components of the application can interact correctly with each other and with the underlying services when executed in a partial but realistic system environment.

5.2.2 Mocking Strategy and Isolation

In order to isolate the test environment from the production environment, we leverage the `firebase-functions-test` package, which allows local testing of Cloud Functions by simulating events within the Firebase Emulator Suite.

5.2.3 Integration Test Cases

User Management

createUser
IT1: Should create a user successfully given valid verification code

- **Precondition**: A valid verification code exists in Firestore for the email.
- **Action**: Call `createUser` function with valid data.
- **Expected result**: User is created in Auth found by email, custom token is returned, and verification code is deleted from Firestore.
- **Outcome**: *Passed*

IT2: Should fail if verification code is missing/invalid

- **Precondition**: Verification code is missing or incorrect.
- **Action**: Call `createUser` function.
- **Expected result**: The function throws an error and user is not created.
- **Outcome**: *Passed*

Path Management

saveUserPath
IT3: Should save a new user path

- **Precondition**: Authenticated user provides valid path data.
- **Action**: Call `saveUserPath` function.
- **Expected result**: A new document is created in `user_paths` with correct data, and a corresponding linked document is created in `paths`.
- **Outcome**: *Passed*

getCommunityPaths
IT4: Should retrieve community paths

- **Precondition**: Community paths exist in the queried region.

- **Action**: Call `getCommunityPaths` function.
- **Expected result**: Returns a list of paths including the previously created one, with correct structure.
- **Outcome**: *Passed*

updateUserPathPrivacy
IT5: Should update user path privacy

- **Precondition**: User has an existing path.
- **Action**: Call `updateUserPathPrivacy` to change visibility.
- **Expected result**: The `isPublic` field in the `user_paths` document is updated to the new value.
- **Outcome**: *Passed*

deleteUserPath
IT6: Should delete user path and orphan path

- **Precondition**: User has a path that is the sole reference to a community path.
- **Action**: Call `deleteUserPath`.
- **Expected result**: The document in `user_paths` is deleted, and the orphaned document in `paths` is also deleted.
- **Outcome**: *Passed*

5.3 System Testing

System testing was performed to validate the end-to-end integration between the iOS client and the Firebase Backend. The primary goal was to ensure that data flows correctly from the device to the cloud and back, satisfying the functional requirements defined in the Design Document.

The testing strategy focused on four main functional areas: Authentication, Path Recording, Data Management, and Community Interaction.

5.3.1 System Test Cases

User Authentication

Sign Up
ST1: Initiated the authentication flow with an unregistered email address

- **Precondition**: App is in unauthenticated state.
- **Action**: Initiate the authentication flow with an unregistered email address.
- **Expected result**: A new user document is created in Firebase, and the app redirects to the Home Screen.
- **Outcome**: *Passed*

Sign In
ST2: Initiated the flow with an existing registered email

- **Precondition**: App is in unauthenticated state with an existing user account.
- **Action**: Initiate the sign in flow with the registered email.
- **Expected result**: The user is successfully authenticated, retrieving their history.
- **Outcome**: *Passed*

Sign Out
ST3: Triggered the sign-out action from the account page

- **Precondition**: User is authenticated and on the account page.
- **Action**: Trigger the sign out action.
- **Expected result**: The local session is cleared, and the user is redirected to the Home Page.
- **Outcome**: *Passed*

Account Deletion
ST4: Selected the permanent delete option from settings

- **Precondition**: User is authenticated and on the settings page.
- **Action**: Select the permanent delete option.
- **Expected result**: The user's authentication record and personal data are removed from the Backend and the app returns to the Welcome Screen.
- **Outcome**: *Passed*

Path Recording

Manual Obstacle Insertion
ST5: While recording, pressed the "Add Obstacle" button

- **Precondition**: User is actively recording a path.
- **Action**: Press the "Add Obstacle" button.
- **Expected result**: A marker is placed on the map, and a visual notification confirms the insertion.
- **Outcome**: *Passed*

Automatic Obstacle Detection
ST6: Simulated a sudden device movement (shake) during recording

- **Precondition**: User is actively recording a path and accelerometer is active.
- **Action**: Simulate a sudden device movement (shake).
- **Expected result**: The accelerometer triggers the "Pothole/Obstacle" logic, automatically logging an event and displaying a confirmation notification.
- **Outcome**: *Passed*

Manual Pause/Resume
ST7: Toggled the pause button during an active ride

- **Precondition**: User is actively recording a ride.
- **Action**: Toggle the pause button.
- **Expected result**: The timer and GPS tracking stop immediately and resume only upon user input.
- **Outcome**: *Passed*

Activity Recognition (Auto-Pause)
ST8: Simulated a stop in cycling movement

- **Precondition**: User is actively recording a ride.
- **Action**: Simulate a stop in movement.
- **Expected result**: The application detects the lack of motion and automatically pauses the recording to preserve data accuracy.
- **Outcome**: *Passed*

User Path Management

Path Updates
ST9: Modified the privacy status and path condition status of an existing entry

- **Precondition**: User checks an existing path entry in their history.
- **Action**: Modify the privacy status (Public/Private) and path condition status.
- **Expected result**: Changes are persisted in the database. Upon reloading the app, the updated status is visible.
- **Outcome**: *Passed*

Path Deletion
ST10: Removed a path from the history

- **Precondition**: User views a path in their history.
- **Action**: Remove the path.
- **Expected result**: The entry disappears immediately from the list and is no longer queryable from the Backend.
- **Outcome**: *Passed*

Start Recording
ST11: Started a recording from an existing path detail view

- **Precondition**: User opens an existing path detail view.
- **Action**: Start a recording.
- **Expected result**: The recording view launches, overlaying the previous path (in a subtle color) on the map to guide the user.
- **Outcome**: *Passed*

Community Path Search

Search Functionality
ST12: Entered specific start and destination points in the community search bar

- **Precondition**: User is on the community search screen.
- **Action**: Enter specific start and destination points.
- **Expected result**: The map populates with public paths matching the criteria uploaded by other users.
- **Outcome**: *Passed*

Community Path Interaction
ST13: Selected a public path from search results

- **Precondition**: Search results are displayed.
- **Action**: Select a public path.
- **Expected result**: The detail view opens. If authenticated, the user can start a new recording using this public path as a reference track.
- **Outcome**: *Passed*

Scenarios

To verify the business logic regarding data aggregation and privacy, we performed specific step-by-step scenarios.

Privacy Consistency Check
ST14: Verify that the "Public/Private" toggle controls data visibility

- **Precondition**: A single community path exists from Bergamo to Milan, connected to a single public user path.
- **Action**: In User History, change the path toggle to **Private** and save. Verify community search for "Bergamo-Milan" no longer shows the path.

- **Expected result**: The path appears in search initially, but is no longer visible after being set to Private, confirming the privacy filter works.
- **Outcome**: *Passed*

Crowd-Sourcing Logic
ST15: Validates how the system aggregates conflicting reports

- **Precondition**: 3 distinct recordings exist for the same route (1 "Impassable", 2 "Excellent"). Search shows "Excellent".
- **Action**: Delete one "Excellent" recording. Repeat the search.
- **Expected result**: The status updates to **Fair** (average of "Impassable" and one "Excellent"), confirming real-time recalculation.
- **Outcome**: *Passed*

5.3.2 Outcome Considerations

All unit tests passed successfully. Thanks to some of these tests, we were able to catch some bugs: for example, thanks to **ST14**, we were able to find out that we were ignoring user privacy in community path browsing, resulting in some private paths appearing in public searches.

6 | Installation Instructions

To distribute BBP on iOS, we utilize Apple's **TestFlight** platform. Depending on the current status of the Apple review process, there are two ways to access the build:

6.1 Option 1: Public Link

If the build has already been approved by Apple, you can join the testing group immediately via the following link:

https://testflight.apple.com/join/AT7aHqTg

6.2 Option 2: Internal Testing (Manual Addition)

If the public link is not yet active, we will need to add you to our internal development team manually. Please follow these steps:

1. **Send your Apple ID email:** Provide us with the email address associated with your Apple account.
2. **Accept the Invitation:** You will receive an email from Apple via App Store Connect. You must click the link inside that email to authorize your device. If you have never logged into App Store Connect you will have to log and accept the terms.
3. **Join the Group:** Once you have accepted the invite, we will add you to the "Internal Testers" group, and you will receive an email with a link to add BBP to your TestFlight dashboard.

6.3 Prerequisites

- An iPhone running **iOS 26.0** or later.
- The **TestFlight** app (downloadable from the App Store).
- A valid Apple ID.

7 | Available acceptance testing infrastructure

We provide two primary environments for executing acceptance tests. While both are suitable for functional verification, physical iOS Device is suggested.

7.1 Physical iOS Devices

The most reliable method for acceptance testing, ensuring the app behaves exactly as it will for the end user.

- **Scope:** Full access to hardware APIs like Motion and Location Sensors and Feedback Generator (Vibration).
- **Network:** Testing under real-world conditions.

7.2 iOS Simulator (macOS)

The iOS Simulator is part of the Xcode toolset. Note that this is a **simulator** (mimicking software behavior) rather than an **emulator** (mimicking hardware).

- **Environment:** Runs natively on macOS.
- **Constraints:**
  - **Hardware:** No support for Motion Sensors and Feedback Generator. Location can be simulated thanks to external apps like *Rocket Simulator*.

7.2.1 Comparison Summary

Feature	Physical Device	macOS Simulator
Hardware Sensors	Fully Supported	Simulated/Unsupported
Touch Gestures	Multi-touch	Mouse

Table 7.1: Comparison between Physical Device and macOS Simulator

8 Effort Spent

	Foad Zahedi Labaf	Lorenzo Fiamingo	Alexander Gabriel Fox
Section 1	1	1	3
Section 2	2	1	3
Section 3	3	3	2
Section 4	1	8	2
Section 5	9	7	4

Table 8.1: The number of hours each group member has worked for this document

9 References

- **Typst** language and its online editor were used to build this document.
- **GitHub** was used for version control and to automate diagram generation.
- **Google Docs** was used for collaborative writing.
- **Xcode** and **SwiftUI** were used to create UI mockups.
- **ChatGPT** and **Grammarly** were used to refine grammar, assist with code development, and automate error checking.